

Django QuerySet والفورم

تقرير أكاديمي شامل عن تقنيات إدارة قواعد البيانات وبناء النماذج

إشراف: المهندس مالك المصنف

ملخص البحث

يستعرض هذا التقرير مفهوم QuerySet في Django وأهميته في إدارة البيانات وتصفية وعرض النتائج، مع أمثلة عملية. كما يوضح طرق إنشاء النموذج (Form) باستخدام HTML ثم ربطه بلغة Python و Django، مع مقارنة عملية بين الطريقتين.

تناول هذا التقرير دراسة مفصلة لـ QuerySet بما في ذلك التصفية، الترتيب، التجميع والإحصاء، بالإضافة إلى المقارنة العملية بين طرق إنشاء نماذج الإدخال باستخدام HTML و Python/Django.

جامعة إب - قسم هندسة البرمجيات

جدول المحتويات

يستعرض هذا التقرير الأكاديمي مفهوم QuerySet في إطار عمل Django وطرق إنشاء النماذج (Forms) باستخدام Python و HTML، مع أمثلة عملية متنوعة للتطبيق في المشاريع البرمجية.

تصفية البيانات	إنشاء QuerySet	أهمية QuerySet	مقدمة عن Django
التجميع والإحصاء	عمليات CRUD	عرض البيانات	ترتيب النتائج
الخلاصة	أمثلة تطبيقية	نماذج Django	نماذج HTML

مقدمة عن Django ORM

Django هو إطار عمل ويب مفتوح المصدر مكتوب بلغة Python، يتبع نمط MVC (Model-View-Controller)، يهدف إلى تسهيل عملية تطوير تطبيقات الويب المعقدة. يتميز بميزات قوية مثل ORM (Object-Relational Mapping) الذي يسهل التعامل مع قواعد البيانات بدون كتابة SQL مباشرة.

ما هو ORM؟

ORM هو تقنية تسمح بالتعامل مع قواعد البيانات العلائقية باستخدام كائنات البرمجة. بدلاً من كتابة استعلامات SQL، يمكنك التعامل مع البيانات باستخدام كائنات Python.

مميزات QuerySet

QuerySet هو مجموعة من العمليات على قاعدة البيانات وتتميز بالتنفيذ المؤجل (Lazy Evaluation)، أي أن الاستعلام لا يتم تنفيذه حتى تحتاج البيانات فعلياً، مما يحسن الأداء بشكل كبير.

فوائد استخدام ORM

يسهل صيانة الكود ويجعله أكثر أماناً، يمكن التبديل بين أنظمة قواعد البيانات المختلفة دون تغيير الكود، ويوفر طبقة حماية ضد هجمات SQL Injection.

مميزات QuerySet ولماذا تحتاجها؟

مميزات أساسية في QuerySet تجعلها الأداة المثالية للتعامل مع قواعد البيانات في Django وتعزيز كفاءة التطوير

قابلية التسلسل وإعادة الاستخدام

يمكنك ربط عدة عمليات معًا وإنشاء استعلامات معقدة بطريقة سهلة وقابلة للقراءة. التوافق مع قواعد بيانات متعددة يسمح بتشغيل نفس الكود على PostgreSQL أو MySQL أو SQLite بدون تغيير.

التنفيذ المؤجل (Lazy Evaluation)

لا يتم تنفيذ الاستعلام فعليًا حتى تحتاج النتائج، مما يسمح بسلاسل الاستعلامات والتحسين التلقائي، وتقليل وقت المعالجة وتحسين أداء قاعدة البيانات.

إنشاء QuerySet في Django

شرح استعمالات وطرق إنشاء QuerySet للتعامل مع قواعد البيانات في Django بطريقة فعالة وسهلة، باستخدام ORM بدلاً من استعمال SQL مباشرة.

أمثلة عملية

```
# استرجاع كل الطلاب
students = Student.objects.all()

# تصفية الطلاب حسب العمر
adults = Student.objects.filter(age__gte=18)

# استبعاد طلاب من مدينة معينة
not_from_ibb = Student.objects.exclude(city='إب')

# الحصول على طالب محدد برقم الهوية
try:
    student = Student.objects.get(id=1045)
except Student.DoesNotExist:
    # التعامل مع حالة عدم وجود الطالب
    pass
```

الطرق الأساسية في QuerySet

()all: استرجاع جميع سجلات النموذج من قاعدة البيانات.
()filter: تصفية السجلات حسب شروط معينة، مع إمكانية استخدام عوامل مقارنة متعددة.
()exclude: استبعاد السجلات التي تطابق الشروط المحددة.
()get: جلب سجل واحد فقط، وإذا وجد أكثر من سجل أو لم يوجد أي سجل يتم إثارة استثناء.

التصفية والاستثناء المتقدم Q objects

العمليات المنطقية المتاحة

| : تمثل عملية OR (أو)

& : تمثل عملية AND (و)

~ : تمثل عملية NOT (نفي)

يمكن دمج هذه العمليات معاً لإنشاء شروط معقدة

مثال للنفي باستخدام ~

للبحث عن الطلاب من جميع المحافظات باستثناء محافظة إب:

```
Student.objects.filter(~Q(city='lbb'))
```

بناء استعلامات ديناميكية

يمكن إنشاء كائنات Q ديناميكياً أثناء تشغيل البرنامج. مثلاً بناءً على مدخلات المستخدم، يمكن إنشاء فلاتر متعددة وإضافتها تدريجياً لبناء استعلام مركب.

عمليات التصفية المتقدمة في Django تتيح لك إنشاء استعلامات معقدة باستخدام كائنات Q. تسمح هذه الكائنات بتنفيذ عمليات منطقية مثل AND و OR و NOT على شروط متعددة، وتعتبر أداة قوية لبناء استعلامات ديناميكية.

```
# استيراد كائنات Q
from django.db.models import Q

# إيجاد الطلاب بعمر أكبر من 20 أو من اليمن : (أو) عملية OR
Student.objects.filter(Q(age__gt=20) | Q(country='Yemen'))

# طلاب من اليمن وبمعدل أعلى من 85 : (و) عملية AND
Student.objects.filter(Q(country='Yemen') & Q(gpa__gte=85))
```

ترتيب النتائج وتخصيص QuerySet

تعرف على طرق ترتيب وتخصيص نتائج الاستعلامات في Django للحصول على أفضل أداء وترتيب منطقي للبيانات المسترجعة

تحسين الأداء باختيار الحقول

`defer('description', 'content')` : تأجيل جلب حقول كبيرة غير مطلوبة فوراً

`only('id', 'title', 'created_at')` : جلب حقول محددة فقط

تحسين أداء العلاقات:

`select_related('author')` : للعلاقات الأحادية (ForeignKey)

`prefetch_related('tags')` : للعلاقات المتعددة (ManyToMany)

هذه الدوال تقلل عدد استعلامات قاعدة البيانات وتحسن الأداء العام للتطبيق

دوال ترتيب وتصفية النتائج

`order_by('field_name')` : لترتيب النتائج تصاعدياً حسب الحقل المحدد

`order_by('-field_name')` : لترتيب تنازلياً

`distinct()` : لاسترجاع النتائج الفريدة بدون تكرار

`reverse()` : لعكس ترتيب النتائج الحالية

مثال:

`Student.objects.order_by('-gpa', 'name')`

`Book.objects.filter(author='أحمد').distinct()`

عرض البيانات بأنماط مختلفة

طرق مختلفة لاسترجاع وتنسيق نتائج الاستعلامات في Django مع مقارنة الخصائص والاستخدامات

()values_list

يسترجع البيانات على شكل مجموعة من ال tuples، أكثر كفاءة من حيث الذاكرة. يفيد في:

- استخراج قائمة مفردة باستخدام flat=True
- معالجة كميات كبيرة من البيانات
- الاستخدامات التي تحتاج أداء عالي

```
# استرجاع كل الأسماء كقائمة
Student.objects.values_list('name', flat=True)
```

```
# النتيجة
['طارق', 'أحمد', 'محمد']
```

()values

يسمح باسترجاع البيانات على شكل قاموس (dict) مع إمكانية تحديد الحقول المطلوبة فقط. يفيد في:

- التعامل مع البيانات ك JSON
- تحسين الأداء بقراءة حقول محددة فقط
- التجميع والإحصاءات مع annotate و aggregate

```
# استرجاع الاسم والعمر فقط
Student.objects.values('name', 'age')
```

```
# النتيجة
[{'name': 'طارق', 'age': 22}, {'name': 'أحمد', 'age': 21}]
```

إنشاء البيانات (Create)

يمكن إنشاء سجل جديد باستخدام الطرق create () أو get_or_create () لتجنب التكرار

```
# إنشاء سجل جديد
Student.objects.create( name="محمد", age=20,
major="هندسة برمجيات" ) # إنشاء أو الحصول على سجل
obj, created =
Student.objects.get_or_create( name="أحمد",
defaults={"age": 22, "major": "علوم حاسوب" } )
```

تحديث البيانات (Update)

يمكن تحديث البيانات بطرق مختلفة منها update () للتحديث الجماعي

```
# تحديث سجل واحد
Student.objects.get(id=1) student.age = 21
student.save() # تحديث مجموعة سجلات
Student.objects.filter(major="هندسة
برمجيات").update( level="متقدم" )
```

حذف البيانات (Delete)

حذف السجلات باستخدام delete () على عنصر واحد أو مجموعة من العناصر

```
# حذف سجل واحد
Student.objects.get(id=5) student.delete() # حذف
مجموعة سجلات
Student.objects.filter(active=False).delete()
```

العمليات على البيانات (CRUD)

يوفر Django QuerySet العديد من الطرق لإجراء عمليات CRUD (إنشاء، قراءة، تحديث، حذف) على البيانات بطريقة بسيطة وسهلة. تتميز هذه العمليات بكونها آمنة وسريعة ومتوافقة مع جميع أنواع قواعد البيانات المدعومة. كما توفر طرق للتعامل مع البيانات بكميات كبيرة (Bulk operations) لتحسين الأداء.

الإحصاء والتجميع

عمليات الإحصاء والتجميع في Django QuerySet تسمح بإجراء حسابات متقدمة على البيانات مثل المجموع والمتوسط والقيم القصوى باستخدام دوال `aggregate` و `annotate`.

أمثلة عملية

```
# حساب متوسط وأعلى درجة لجميع الطلاب
from django.db.models import Avg, Max, Count, Sum
result = Student.objects.aggregate(
    avg_grade=Avg('grade'),
    max_grade=Max('grade')
)
# النتيجة: {'avg_grade': 85.5, 'max_grade': 98}

# إضافة حقول مشتقة لكل قسم تعليمي
departments = Department.objects.annotate(
    student_count=Count('student'),
    total_grades=Sum('student__grade')
)

# حساب عدد الكورسات لكل طالب
students = Student.objects.annotate(
    course_count=Count('courses')
).filter(course_count__gt=3)

# التجميع المركب - المتوسط حسب المدينة
from django.db.models import Q, F
cities_data = Student.objects.values('city').annotate(
    avg_grade=Avg('grade'),
    student_count=Count('id')
)
```

الفرق بين `aggregate` و `annotate`

aggregate(): تطبيق وظائف الإحصاء على كامل مجموعة النتائج وإرجاع قيمة واحدة، مثل حساب متوسط الدرجات لجميع الطلاب.

annotate(): إضافة حقول مشتقة لكل سجل في النتائج، مثل حساب عدد المواد لكل طالب وإضافته كحقل للنتيجة.

الدوال المتاحة:

- **Sum:** مجموع القيم
- **Avg:** متوسط القيم
- **Count:** عدد السجلات
- **Max/Min:** أقصى/أدنى قيمة
- **StdDev:** الانحراف المعياري
- **Variance:** التباين

مقارنة: form بـ HTML و بـ Django

مقارنة بين استخدام النماذج التقليدية في HTML والنماذج المدمجة في إطار عمل Django من حيث الميزات والتحديات

نماذج Django (ModelForm)

- ✓ توليد تلقائي للحقول من نموذج البيانات
- ✓ تحقق تلقائي من المدخلات
- ✓ حماية تلقائية من هجمات CSRF
- ✓ تنسيق تلقائي للبيانات (قبل وبعد الحفظ)
- ✓ تقليل حجم الكود وزيادة الإنتاجية
- ✗ أقل تحكمًا في الـ HTML المُولّد
- ✗ منحى تعلم أولي للإعدادات المتقدمة

نماذج HTML التقليدية

- ✓ تحكم كامل في الشكل والتنسيق (HTML/CSS)
- ✓ تعمل بشكل مستقل عن إطار العمل
- ✓ سهولة الدمج مع أي تقنية خلفية
- ✗ تتطلب كتابة كود التحقق يدويًا
- ✗ كود إضافي لمعالجة البيانات
- ✗ تكرار الكود عند إنشاء نماذج مشابهة
- ✗ احتمالية أكبر للأخطاء الأمنية

استقبال البيانات باستخدام Python

يمكن معالجة البيانات المرسلة من النموذج باستخدام Python بطريقتين: إما مباشرة باستخدام إطار عمل مثل Flask، أو من خلال Django. هذا مثال باستخدام Flask:

```
from flask import Flask, request, render_template app =
Flask(__name__) @app.route('/register', methods=['GET',
'POST']) def register(): if request.method == 'POST': #
student_name = request.form['student_name'] student_id =
request.form['student_id'] email = request.form['email']
department = request.form['department'] # معالجة البيانات
print(f"تم استلام بيانات") (مثلاً: حفظها في قاعدة بيانات)
return "تم التسجيل بنجاح!" # عرض النموذج
return render_template('register_form.html') if __name__ ==
'__main__': app.run(debug=True)
```

تنفيذ عملي: فورم HTML بسيط

يمكن إنشاء نموذج بسيط باستخدام HTML لجمع بيانات المستخدم. سنقوم بإنشاء نموذج تسجيل الطلاب يحتوي على حقول مختلفة، ثم نرى كيف يمكن استقبال هذه البيانات ومعالجتها باستخدام Python.

```
<!-- بسيط لتسجيل طالب HTML نموذج --> <form method="POST" action="/register"> <div> <label
for="student_name">اسم الطالب:</label> <input type="text" id="student_name" name="student_name" required>
</div> <div> <label for="student_id">الرقم الجامعي:</label> <input type="text" id="student_id" name="student_id"
required> </div> <div> <label for="email">البريد الإلكتروني:</label> <input type="email" id="email" name="email">
</div> <div> <label for="department">القسم:</label> <select id="department" name="department"> <option
value="cs">علوم حاسوب</option> <option value="it">تقنية معلومات</option> <option value="se">هندسة
</option> </select> </div> <button type="submit">تسجيل</button> </form>
```

تنفيذ عملي: فورم باستخدام Django ModelForm

استخدام نظام النماذج المدمج في Django لإنشاء فورم متكامل من النموذج (Model) بشكل مباشر دون الحاجة لتحديد الحقول يدويًا، مما يوفر الوقت ويقلل الأخطاء.

إنشاء النموذج والاستخدام

```

# models.py
from django.db import models

class Student(models.Model):
    name = models.CharField(max_length=100, verbose_name="اسم الطالب")
    student_id = models.CharField(max_length=10, unique=True, verbose_name="الرقم الجامعي")
    email = models.EmailField(verbose_name="البريد الإلكتروني")
    level = models.CharField(max_length=1, choices=LEVEL_CHOICES, verbose_name="المستوى الدراسي")
    gpa = models.DecimalField(max_digits=3, decimal_places=2, verbose_name="المعدل التراكمي")
    enrollment_date = models.DateField(verbose_name="تاريخ التسجيل")

    def __str__(self):
        return f"{self.name} ({self.student_id})"

# forms.py
from django import forms
from .models import Student

class StudentForm(forms.ModelForm):
    class Meta:
        model = Student
        fields = '__all__' # استخدام جميع الحقول من النموذج
        # يمكن تحديد حقول معينة: fields = ['name', 'email', 'level']
        widgets = {
            'name': forms.TextInput(attrs={'class': 'form-control'}),
            'enrollment_date': forms.DateInput(
                attrs={'type': 'date', 'class': 'form-control'}),
        }

# views.py
from django.shortcuts import render, redirect
from .forms import StudentForm

def add_student(request):
    if request.method == 'POST':
        form = StudentForm(request.POST)
        if form.is_valid():
            form.save() # حفظ البيانات مباشرة في قاعدة البيانات
            return redirect('student_list')
    else:
        form = StudentForm()

    return render(request, 'students/add.html', {'form': form})

```

نموذج ModelForm متكامل

```

# models.py
from django.db import models

class Student(models.Model):
    LEVEL_CHOICES = [
        ('1', 'المستوى الأول'),
        ('2', 'المستوى الثاني'),
        ('3', 'المستوى الثالث'),
        ('4', 'المستوى الرابع'),
    ]

    name = models.CharField(max_length=100, verbose_name="اسم الطالب")
    student_id = models.CharField(max_length=10, unique=True, verbose_name="الرقم الجامعي")
    email = models.EmailField(verbose_name="البريد الإلكتروني")
    level = models.CharField(max_length=1, choices=LEVEL_CHOICES, verbose_name="المستوى الدراسي")
    gpa = models.DecimalField(max_digits=3, decimal_places=2, verbose_name="المعدل التراكمي")
    enrollment_date = models.DateField(verbose_name="تاريخ التسجيل")

    def __str__(self):
        return f"{self.name} ({self.student_id})"

```

الخلاصة والتوصيات

أهم النقاط المستفادة:

- تُعتبر Django QuerySet أداة قوية تسهل التعامل مع قواعد البيانات بطريقة برمجية خالصة
- تعزز الأمان وتوفر الوقت من خلال تجنب كتابة استعلامات SQL المباشرة
- تقدم الفورم في Django حلاً متكاملًا لمعالجة البيانات مع التحقق من الصحة تلقائيًا
- استخدام ModelForm يزيد من كفاءة الربط بين النماذج والبيانات

التوصيات:

- الاستفادة من خاصية التكامل بين QuerySet والفورم لبناء تطبيقات متكاملة
- استخدام الميزات المتقدمة مثل annotate و aggregate في التحليلات المركبة
- استخدام الفورم في Django لتبسيط عمليات الإدخال وضمان سلامة البيانات

إعداد الطالب: طارق العمري